

research

Phase 0 Research — Chapter Hierarchy

Feature: specs/003-chapter-hierarchy | **Date:** 2026-09-05

Every decision below is labelled D-CH-n and carries the measurement that produced it. Where a question could not be settled from evidence in the tree, it appears in the **Unverified register** at the end rather than being answered by a plausible guess. One entry in that register blocks the whole feature.

D-CH-1 — The sort order is a measurement, not an argument

Question: given the dotted id, what order do chapters appear in, and does a parent land beside its children?

Measured, by running it rather than reasoning about it:

```
printf '10\n02.09\n01\n03\n02\n02.10\n02.01.01\n02.01\n' |
LC_ALL=C sort
```

Position	1	2	3	4	5	6	7	8
id	01	02	02.01	02.01.01	02.09	02.10	03	10

And the byte values that produce it:

character	byte
.	0x2E
0	0x30

character	byte
9	0x39

Decision: ordering is **byte-lexicographic on the id string**, and a parent sorting immediately before its first child is a **consequence** of . sorting below every digit — not a rule anybody implements.

Three things this table settles that an argument would not have:

1. **02.01.01 sorts before 02.09.** A reader reasoning about “depth” would expect grandchildren to come after all of 02’s direct children. They do not, and the order is still correct — it is a depth-first walk, which is what a reader of a table of contents expects.
2. **02.10 sorts after 02.09.** This is the case that distinguishes a padded-string comparison from a decimal one. Both orders agree here; they disagree at 02.9, which the grammar rejects.
3. **10 sorts last, not second.** Two-digit padding is what makes this true. A corpus reaching 100 needs three-digit padding from the start, which is why the grammar is `[0-9]{2,}` rather than `[0-9]{2}`.

The load-bearing consequence, and it is not about sorting at all: the order is correct **only because every component is zero-padded**. 02.2 sorts *after* 02.10 and is wrong in a way no sorting test with well-formed input can detect. **The validator, not the comparator, is what makes the ordering true.** This is why the specification gates the grammar (SC-002) and forbids touching the comparator (FR-011).

Already implemented, and correct. `cmd/workshop-server/main.go:2126` reads `sort.Slice(out, func(i, j int) bool { return out[i].ID < out[j].ID })`. The safest action on the sorting code is **no action**; the assertion goes on the ordering (H4), so a future rewrite into a numeric comparator is caught by a gate rather than by a reader noticing 02.10 in the wrong place.

D-CH-2 — The four embeddings: where a dotted id has to survive, and whether it does

Question: the id is embedded in four independent layers, each written without a dot in mind. A scheme that breaks any of them is not viable. Which break?

Measured, 2026-09-05, one layer at a time. None breaks.

Embedding 1 — the filesystem path

SafeSlug (pkg/curriculum/curriculum.go:205) is an **allowlist**, and . is in it:

```
case r >= 'a' && r <= 'z', r >= 'A' && r <= 'Z', r >= '0' && r <= '9':
case r == '-', r == '_', r == '.':
```

It rejects . and .. as whole strings, bounds the length at 128, and rejects / and \ at :209. ChapterDir (:187) then tries chapter-<slug> and then <slug>, so chapter-02.01 resolves with no change.

Verdict: SURVIVES, with no edit. And the rejection at :209 is the strongest single argument against the 02/01 alternative — see D-CH-3.

Embedding 2 — the HTTP route

Six routes register a {chapter} wildcard:

main.go:547, :817, :818, :819, :841, :843. Go's {name} wildcard matches **exactly one path segment**, and . is an ordinary character within a segment — there is no dot delimiter in the pattern language, unlike some frameworks where . ends a match or introduces a format suffix.

Verdict: SURVIVES, with no edit.

Embedding 3 — the client URL

core/api.ts:107, :122, :155 and others build paths as `/api/chapters/\${encodeURIComponent(slug)}`. . is an **unreserved** character and encodeURIComponent leaves it as-is, so 02.01 travels literally and matches the wildcard.

Verdict: SURVIVES, with no edit.

Embedding 4 — the search catalogue's filename-shape detector

This one was checked because a false positive here would raise a content-boundary alarm on **every** sub-chapter id, and that is the kind of failure that gets “fixed” by adding an exemption.

HasSourceFilenameShape (pkg/search/catalog.go:368) compiles its pattern from a **closed** extension list (:320–:323) plus one archive-chunk alternative (:338):

```
SourceFilenameShapeExtensions = []string{
    "mp4", "mov", "mkv", "webm", "m4v", "avi", "mp3", "wav",
    "tar.gz", "tgz", "zip", "7z", "sha256",
}
sourceFilenameShapePartFragment = "part" + `~?\d{1,5}`
```

No entry is numeric-only, and the archive alternative requires the literal part. So `.01` is matched by no alternative and `02.01` is not read as a filename.

Verdict: SURVIVES — and it survives by a property of the list, which is one edit from changing. An assertion is added rather than a comment, because “the list happens not to contain a numeric entry” is a fact about today’s list.

What the four embeddings settle

Four of the five layers a dotted id must survive already survive it, none by design and all by measurement. The fifth — derivation, `ordinalOf` and everything downstream — does not, and that is where the entire implementation cost of this feature sits.

This changes the shape of the work materially. The instinctive framing, “*add hierarchy support to the platform*”, implies a change to identity, routing and storage. The measurement says the change is **one new grammar, one derivation replacement, and a set of frozen literals**. Anything larger is a rewrite of code that is already correct, and rewriting correct code is how the four surviving layers would stop surviving.

D-CH-3 — Why chapters/02/01 was rejected

Question: path segments read more naturally as a hierarchy than a dotted string does. Why not use them?

Measured: it requires `/` inside the chapter slug. `SafeSlug` rejects `/` and `\` at `curriculum.go:209`, and that rejection is a **path-traversal boundary** — the function’s own comment describes it as an allowlist chosen because “*the interesting inputs are the encoded and the exotic ones*”. Accepting `/` to support a naming preference trades a security property for aesthetics.

And it does not work anyway. Go’s {chapter} wildcard matches one segment. GET /api/chapters/02/01 would not match GET /api/chapters/{chapter} at all, so all six routes would need a multi-segment form ({chapter}...), which changes their matching semantics for every existing chapter and reopens the traversal question at the routing layer as well as the filesystem one.

Decision: rejected. Two independent mechanisms — one security, one routing — would have to be weakened, and the only thing gained is that the id looks more like a tree in a URL bar.

D-CH-4 — Why a flat id with a stored parent_id was rejected

Question: the conventional relational answer is an opaque flat id plus a parent_id column. It is what most systems do. Why not here?

Measured, on the mechanism rather than on taste. A stored parent is a **second source of truth about structure**. The failure it enables is not an exception:

a chapter whose id says 02.01 and whose stored parent says 03 is **not a detectable error state**. It is two facts, both readable, both plausible, with no symptom until something navigates — and then the navigation *succeeds*, into the wrong chapter.

It also loses the ordering property from D-CH-1. With an opaque flat id, byte order over ids carries no structure at all, so the sort must be reconstructed by walking parent links. Filesystem order and API order then agree only if a sorting step keeps them agreeing, where today they agree by construction. H4 becomes an assertion about two algorithms instead of a property of one string.

Decision: rejected. The argument *for* it is cost — derivation repeats work — and the work is a string split over 3 ids. The argument against is a class of bug with no symptom.

Recorded because it will be re-proposed. It arrives disguised as a performance improvement (“cache the child list”) or as normalisation hygiene (“ids should not be meaningful”). The second is a real principle and it is the right one for *content* identifiers, which is why passages,

areas, terms and questions all carry minted ULIDs. A chapter id is a **structural name under an operator's control**, not a content identifier, and the distinction is recorded in `data-model.md §1.2`.

D-CH-5 — The root cause is in a specification, and it cannot be fixed by code

Question: `ordinalOf("02.01")` returns 2. Where is the defect?

Measured: not in `ordinalOf.specs/001-workshop-curriculum-platform/data-model.md:45` types `Chapter.ordinal` as `int`:

```
| `ordinal` | int | zero-padded on disk (`chapters/01/`), matching  
existing convention |
```

`internal/api/chapters.go:736` is the faithful implementation of that type — it consumes leading digits and stops — and `chapterTitle(:755)` is the faithful consumer of *that*. **Under an `int ordinal` there is no correct implementation for `02.01`**, so every ordinal collision in this feature is that one line reaching the surface, three call sites away.

A second contradiction surfaced with it, in the same feature's contracts. `specs/001-.../contracts/http-api.md:60` reads:

```
{chapter} | chapter slug ... The zero-padded ordinal is not  
accepted as a path key — one key, one meaning.
```

`while platform/gates/route-manifest.tsv:72` declares the live route `GET /api/chapters/01` — and `01` **is** the zero-padded ordinal. **The contract and the implementation have disagreed since before this feature existed**, and the disagreement was invisible for a structural reason: with one chapter, the slug and the zero-padded ordinal are the same string. They stop being the same string the moment `ordinal` is not an `int`.

Decision: this is Phase 0, it is an operator decision, and it blocks everything. Both are inherited defects in published artifacts; neither was introduced here; neither may be inherited silently by writing an implementation that presumes an answer.

Why the apparently safe option is not safe. Three ways out were considered:

Option	Diff size	What it actually does
widen to an ordered ordinal_path	one wire field, one derivation	representable ordering at every depth makes every sub-chapter permanently unorderable and unlabelled — it names the problem and moves on. The front end's existing null-versus-2 disagreement becomes the <i>contracted</i> behaviour
keep int, sub-chapters have no ordinal	zero	renumbers every existing chapter the first time a sub-chapter is inserted, breaking every deep link, every registry scope and every archived artifact
keep int, flatten sub-chapters into one number space	small	

The zero-diff option looks like the conservative one and is the only one that makes the defect permanent. That is recorded here because “change nothing” is the default an unexamined decision falls into.

D-CH-6 — Why the redaction defect needs a two-scope fixture, measured

Question: cmd/workshop-redact/main.go:616 is now `*rec.Scope != chapter, fixed in 266f443 from strings.HasSuffix(*rec.Scope, chapter)`. What test guards it?

Measured, on the operators rather than on the code. For a chapter argument of 01:

registry scope	HasSuffix(scope, "01")	scope == "01"	agree?
01	true	true	yes

registry scope	HasSuffix(scope, "01")	scope == "01"	agree?
02.01	true	false	no
docs	false	false	yes

On a registry whose only chapter scope is 01, the two operators return the same answer for every input. The narrowing fix is a **no-op** there. Every existing test therefore passes with the old code and with the new code, which means the fix is currently protected by nothing at all.

Decision: the guard is a property of the **fixture shape**, not of the assertion. The test needs a registry holding at least two scopes, one a sub-chapter of the other, and the paired proof needs **two** cases:

1. two-scope fixture + HasSuffix mutation $\Rightarrow$ the test must go **red**;
2. one-scope fixture + the same mutation $\Rightarrow$ the test must stay **green**.

The second case is unusual — it asserts that a test *fails to detect* something — and it is the one that matters, because without it a maintainer who simplifies the fixture sees everything still pass and concludes the two-scope shape was incidental.

Why this is first in the phase order. The output of this path is a **disclosure-control record** — a document whose entire purpose is to state what was covered. A coverage claim that silently overstates itself is worse than a pipeline that produces nothing, because an absence gets noticed.

D-CH-7 — The two widenings are widenings, and both directions must be proved

Question: `meeting_notes.py:109` and `author.py:159` were widened in 266f443. What does their proof have to assert?

Measured, the current patterns:

```
# pipeline/extract/meeting_notes.py:109
_TRANSCRIPT_PATH_RE = re.compile(r"^(chapter-(\d+(?:\.\d+)*))/transcript\.md$")

# pipeline/extract/author.py:159
_ELIGIBLE_SOURCE = re.compile(r"^chapter-\d+(?:\.\d+)*/(transcript\.md|exercise-\d+\.md)$")
```


Both were `chapter-\d+`, and both call sites skip a non-match with `continue`. So before the fix, a sub-chapter was dropped **silently, on a green run** — no meeting notes extracted, and never a citable authoring source.

Decision: each paired mutation must assert **both** directions —

- **(a)** revert to `chapter-\d+`: the assertion that `chapter-02.01/transcript.md` matches must fail;
- **(b)** narrow to `chapter-\d+\. \d+`: the assertion that `chapter-01/transcript.md` matches must fail.

Why (b) is not redundant. A proof asserting only (a) passes against a pattern that has stopped matching the flat form — which would be a *worse* defect than the original, silent in exactly the same way, and introduced by the fix for it. The commit message for 266f443 records this reasoning independently: *“both directions asserted, because a widening that stopped matching the old form would be worse than the bug it fixed.”*

Note what is deliberately NOT widened. `author.py:159` keeps `exercise-\d+\.md` flat. Exercise numbering is not a chapter path and has no hierarchy; widening it because it sits in the same regex would be a change with no reason behind it.

D-CH-8 — The silent-content defects, ranked by what they produce

Question: several frozen `chapter-01` literals remain. They are not equally dangerous. Which first?

Measured, by what each produces when it is wrong:

Location	Frozen thing	What a wrong run produces
<code>scripts/ingest.sh:93</code>	<code>WORKSHOP_TRANSCRIPT_JSON</code> defaults to <code>chapter 01's transcript JSON</code>	chapter 01's content ingested under another chapter's scope. Every downstream artifact then

Location	Frozen thing	What a wrong run produces
scripts/ingest.sh:46	CHAPTER="01"	agrees with itself. This is the worst outcome in the feature a run with no chapter argument ingests chapter 01 — recoverable, but indistinguishable in the output from an explicit choice no new chapter ever reaches /api/suggest, and because the file is compiled in, no amount of adding content changes it without a rebuild
pkg/search/suggest-sources.json:10,34,57,77,101,125	six chapter-01/... rows, //go:embed-ed at catalog.go:70–71	every deep link in a new chapter's knowledge artifacts points into chapter 01
chapter-01/knowledge/build.py:363, :435	/chapters/01/transcript#p-%s deep links	new chapters' artifacts labelled as chapter 01's
chapter-01/knowledge/build.py:478, :564, :582, :592	"chapter_slug": "01" literals	a transcript built without explicit arguments is labelled chapter 01
pipeline/build_transcript.py:143–144	default="chapter-01", default="Chapter 1"	calibration measures chapter 01
pipeline/calibrate.sh:36	a frozen absolute path to chapter 01's recording	

Location	Frozen thing	What a wrong run produces
		regardless of which chapter was asked about

Decision: the ingest transcript default is fixed first within its phase, and the fix is a **refusal**, not a better default. Naming a chapter and omitting the transcript is an incomplete request, and the honest answer to an incomplete request is to say what is missing.

One line in this list is already right, and it is the model for the rest.

chapter-01/knowledge/build.py:415 reads "chapter_slug": r["scope"] — derived from the record being processed. The four literals in the same file are the same value, hardcoded, four lines apart from the correct form. That is what makes this file the clearest example in the feature: the right answer is already present in it.

D-CH-9 — The precedent for a gate that does not hardcode a chapter

Question: every new gate needs a chapter id. Where does it come from?

Measured: platform/gates/verify-absence-honesty.sh:127–134 already solves this:

```
# A chapter that exists, taken from the live tree rather than
# hardcoded.
CH="$(curl -s --max-time 20 "$BASE/api/chapters" \
  | sed -n 's/.*"id"[[:space:]]*:[[:space:]]*"([A-Za-
z0-9._-]*)"/.*\/1/p' | head -1)"
if [ -z "$CH" ]; then
  undet "A2 no chapter could be read from the live tree; §3.12
  was NOT graded live"
fi
```

Two properties are worth copying and are named so they are copied deliberately: it takes the chapter from GET /api/chapters, and **failing to get one is undet** — **could-not-determine** — **not a pass and not a failure**. Its character class already includes ., so it reads a sub-chapter id correctly today.

Decision: every gate this feature adds follows this shape (FR-035, G-CH-14). The contrast is `route-manifest.tsv:64, #subst {chapter} 01`, where a single hardcoded value means **the manifest cannot exercise a hierarchy at all** — one substitution cannot produce a parent and a child.

Unverified register

An entry here is not a minor caveat. The first one blocks the feature.

- **U1 — the ordinal type and the path-key contradiction (D-CH-5). BLOCKING.** Both are amendments to published feature-001 artifacts (`data-model.md:45`, `contracts/http-api.md:60`) and both are operator decisions. Every task depending on `ordinal_path` carries a `[BLOCKED: ordinal type]` marker. **No implementation may settle this by shipping.**
- **U2 — whether GET /api/chapters should be brought onto the three-state envelope.** It currently returns a bare `{"chapters": ...}` at 200 and `{"error": ...}` at 500 (`main.go:2085–2091`); it does not call `writeUnavailable`. FR-028 requires the filters echo on every status *including* `could-not-determine`, which presumes the envelope. Whether that is in scope here or is a separate correction to the 001 contract is **not settled**, and the contract's §8 U2 records it as target shape rather than as description.
- **U3 — the maximum depth this corpus will reach.** Measured at 2 today. H6 forbids asserting a limit; nothing here predicts one.
- **U4 — whether any consumer outside this repository reads Chapter.ordinal.** The front end does (`models.ts:301, :357`) and is changed with it. Nothing else was found, but the search covered this repository only, and a public artifact carrying the field could exist outside it. Recorded because A1 **removes** the field rather than deprecating it.
- **U5 — whether 02 and 02.01 will ingest cleanly once the pipeline accepts them.** Both are archived and hash-verified; **neither has a transcript** and neither holds a single passage. Nothing in this feature runs them through the pipeline, and no claim is made here about what will happen when something does.